Project Meridian

Design Decisions and Evaluation
Avathon Take-Home Assignment
Jason Peng
November 5, 2026

What Meridian is

Meridian is a local chat app for asking engineering questions about the OpenLPBF v2, an open-source metal 3D printer. An engineer can ask something like “what is the recoater plate made of?” and get an answer that cites the drawing or parts-list row it came from. When the answer is about a specific part, its drawing opens next to the chat, along with a 3D model reconstructed from that drawing when one exists.

The data is 30 part drawings and a bill of materials (BOM) with 192 rows, and it’s clear the two were made separately. A drawing might use a part’s Danish name, or list a specific aluminium alloy where the BOM just says “Aluminium.” Eight of the drawings are scans with no readable text, and one drawing’s title field just says “Description” because the designer never filled it in. Most of the design comes down to handling mismatches like these so an engineer can see what each source actually says.

Handling those mismatches takes careful interpretation, and interpretation is slow and expensive. So I split the system into two stages. A preparation stage does all the interpretation up front. It reads every file, pulls out the facts, links each drawing to its BOM row, and builds the 3D models. The chat app then answers questions from those prepared records, so each question is a quick lookup instead of a fresh read through 30 PDFs.

With 48 hours, I focused on getting one complete path working. An engineer asks a question, gets a cited answer with the drawing and 3D model next to it, and can correct anything that’s wrong. This report follows that path in order: how the data is prepared, how chat answers questions, how the 3D models are built, how corrections work, and how I tested the whole thing.

Preparing the data

The preparation stage turns 30 drawings and a spreadsheet into one clean record per part. Much of that work is mechanical and can be done with ordinary code, like pulling text out of a PDF or parsing a price. Some of it needs real judgment, though. Recognizing that “Cylinder Spænd Ring” is Danish for cylinder clamp ring, or working out which number on a drawing is the thickness, isn’t something a fixed rule can do reliably across every drawing.

The proposer–reviewer pipeline

For the steps that need judgment, I use a two-model pipeline loosely modeled on actor-critic methods. One model, the proposer, makes a specific claim about the data, and a second model, the reviewer, checks that claim against the original source. GPT-6.1 Sol is the proposer and Claude Opus is the reviewer. For example, the proposer might claim that the drawing titled "Cylinder Spænd Ring" belongs to the BOM row "Build cylinder clamp ring," and the reviewer would look at both and decide whether the evidence supports it.

The main reason for this design is scale. Rules work well on clean data, and I use them wherever I can, but without the pipeline every messy case would need either its own special rule or a person's time, and neither grows well as the dataset grows. With the pipeline, the models handle the interpretation, and a person only sees the cases the two models can't settle.

The same pipeline is also what makes it possible to generate 3D models automatically. Building a model from a drawing mostly comes down to reading the drawing accurately, and that reading is exactly what the pipeline does. Once the reviewer confirms the proposer's reading, the dimensions go straight into the CAD code, and nobody has to open CAD software. The 3D section below covers this in more detail.

Splitting the work this way also makes disagreements useful. Because the reviewer is checking one specific claim, it can point to the exact value it thinks is wrong. The proposer gets one retry with that feedback, and if the reviewer still disagrees, the item goes to a person along with both models' reasoning. I limited it to one retry because a longer back-and-forth tends to end with one model simply giving in. Every proposal and review is saved, so anyone can later see how a value was accepted.

Reading the drawings

Each drawing has a title block, a box in the corner of the sheet with labeled fields for the part's title, material, weight, and notes. Most of what a drawing says about a part, apart from its dimensions, lives in the title block, so reading it correctly matters more than anything else on the sheet.

On the 22 clean drawings, the text can be pulled straight from the PDF, but it comes out in whatever order the PDF happens to store it. In early tests, that made some titles come out as a stray grid letter from the page border, or as the designer's name. To fix this, the parser looks for each printed label first, like MATERIAL, and then reads the text in the spot where that field's value is written. As a bonus, this records where on the page each value came from, which later lets a citation in chat open the drawing with that exact field highlighted.

The other eight drawings are scans, which are just images with no text to pull out. A quick test with standard OCR software garbled the Danish characters, so the scans go through the pipeline instead. The proposer reads the title block from the image and copies each field exactly as printed, and the reviewer checks that reading against the same image. On scans, only the title block is read, since the drawing itself is too blurry to measure from, so scans don't get

3D models.

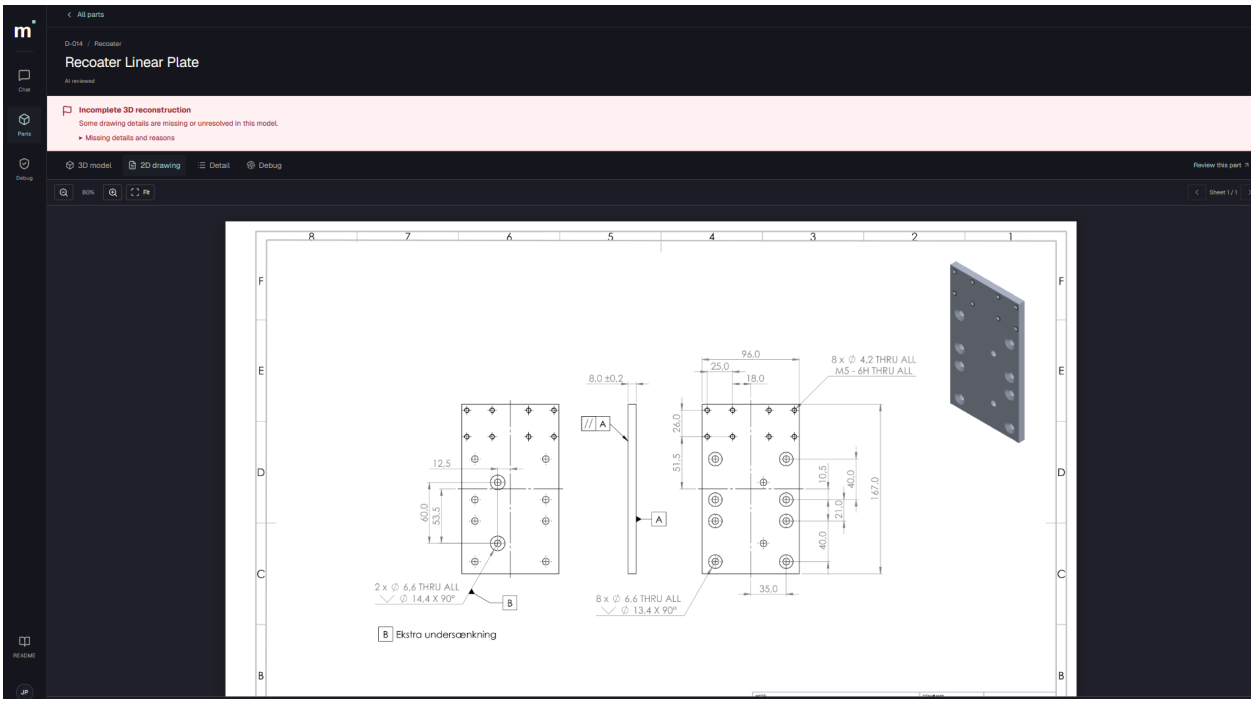


Figure 1. The Recoater Linear Plate's source drawing on its part page, where any extracted value can be checked against the sheet.

Cleaning the BOM

The BOM is a spreadsheet listing every part in the machine, with columns for things like supplier, cost, and material. It has the usual spreadsheet mess, like prices stored as text and a few misspelled status values. That kind of mess is predictable, so it's cleaned with fixed rules, and every cleaned value keeps the original cell next to it.

One problem was easy to miss. Seventy-five rows have no unit price, but every one of them still shows a total of DKK0.00. Adding up that column would make the machine look much cheaper than it is, with no hint that prices were missing, so the cleanup leaves those totals blank.

Linking drawings to the BOM

The drawings and the BOM each know different things about a part. A drawing has its dimensions and weight, while the BOM has its supplier and cost. To answer a question like "what is the recoater plate made of, and who supplies it?", the app needs to know which drawing and which BOM row describe the same part.

That turned out to be the hardest part of preparation. In an early test, simple name matching only got about 11 of 26 drawings clearly right. Most of the misses were Danish titles or different wording, and some were genuinely ambiguous, like a file named Door_x2 that could

be either of the machine's doors. So code handles the exact name matches, and the pipeline handles the rest, using the drawing's title, notes, and material as evidence.

Once a drawing and a BOM row are linked, they sometimes both give a value for the same field, usually the material, and they don't always use the same words. When that happens, code decides what to show. If the drawing says EN-AW 5005 and the BOM says Aluminium, the app shows EN-AW 5005, since that's a more specific name for the same thing. If the two sources genuinely contradict each other, the app shows the value as Undetermined, which is its label for any value it can't settle, and shows both versions instead of picking one.

Knowing that EN-AW 5005 is an aluminium is general engineering knowledge, though, and nothing in the dataset says so. The pipeline proposes material relationships like this one, and a person approves each before it's used, because one wrong mapping would affect every part made of that material. Once a mapping is approved, a search for aluminium parts finds the EN-AW 5005 parts too.

Answering questions in chat

With the data prepared, chat's job is to find the right facts and explain them clearly. The chat model never reads a PDF. Instead, it calls lookup tools that search the prepared records and return facts along with where each one came from. Before the model ever sees a fact, code has already decided what it should display, using the rules from the linking section. That leaves the model with two jobs, choosing the right tools and writing a clear answer, and it means two people asking the same question get the same facts.

Missing information is handled one field at a time, so a single gap doesn't make a whole part unknown. If a drawing specifies an M5 threaded hole but no screw torque, the torque is Undetermined, and the hole spec is still given as a known fact.

For an answer to be checkable, the app needs to know which sentence came from which source, so the model marks its answer as it writes. After each claim, it adds a citation marker naming the fact it used, and the app turns that into a small tag that opens the drawing at the cited spot. Where a value is missing, the model adds an Undetermined marker instead, which the app shows as a tag explaining why the value is unknown. The parts an answer cites also decide which drawings and 3D models appear next to it.

Past conversations are saved, but each new question pulls current records, so a correction approved today applies to every question asked afterward.

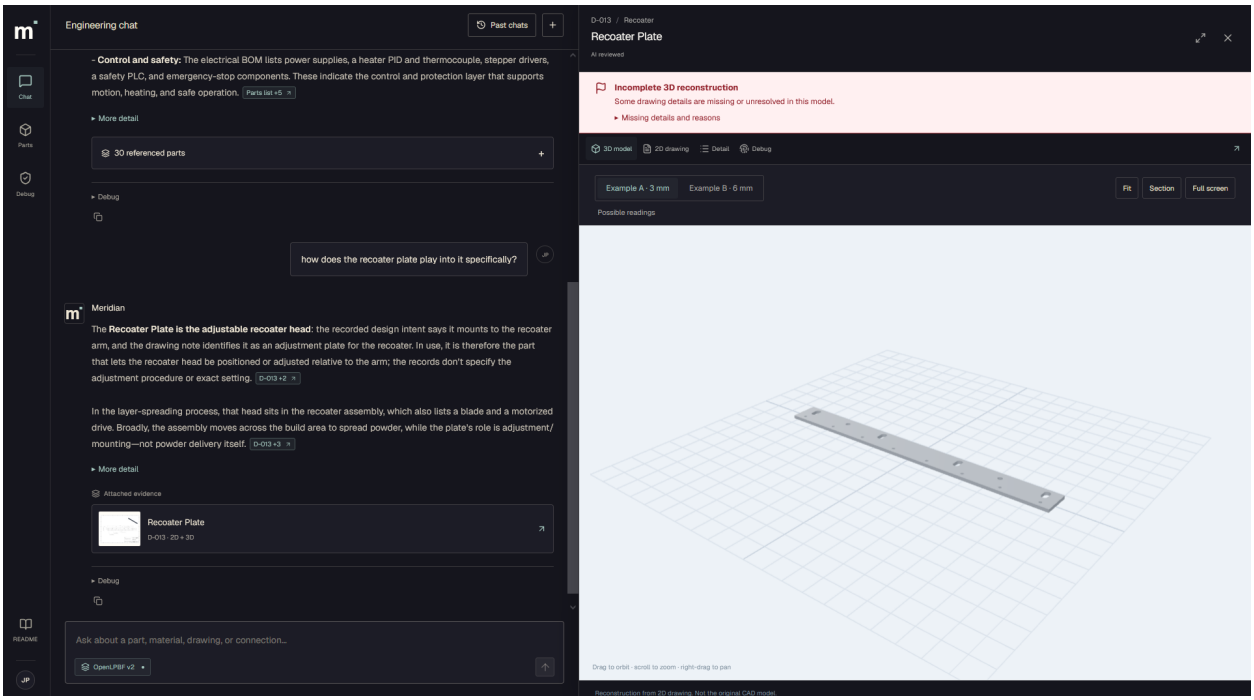


Figure 2. An answer about the Recoater Plate, with its 3D model in the side panel. Both come from the same part record.

Building the 3D models

A 3D model lets an engineer see a part's shape at a glance instead of working it out from a technical drawing. Building these by hand would take a CAD engineer a long time per part, so I generate them from the drawings automatically.

To keep that reliable, the models come from a template, which is a fill-in-the-blanks description of a part's shape, instead of free-form CAD code. The flat-part template covers plates, rings, and discs with holes cut into them, which accounts for most of the simpler parts in this machine. The proposer fills in the template from the drawing, the reviewer checks it, and a Python CAD library called CadQuery turns the confirmed template into a solid. Chat reads its dimensions from the same template, so the thickness quoted in an answer always matches the model.

Some drawings don't give a single clear answer. On a few parts, the thickness is printed as two numbers stacked on top of each other, like 6.0 over 3.0 on the Recoater Plate, and the drawing doesn't say whether that means two versions of the part or a step within one part. Rather than guess, the generator builds a model for each reading, and the 3D view lets you switch between them. Similarly, when a small feature can't be placed, like holes with a known size but no stated spacing, the model is built without it and the viewer notes what's missing.

In total, the generator produced models for nine parts, or twelve versions counting the alternate thicknesses. Some are complete, like the Recoater Linear Plate with all eighteen of its

countersunk holes. Others are partial. The Recoater Plate drawing calls for eight small threaded holes, but its dimensions only work out for two of them, so only those two are modeled.

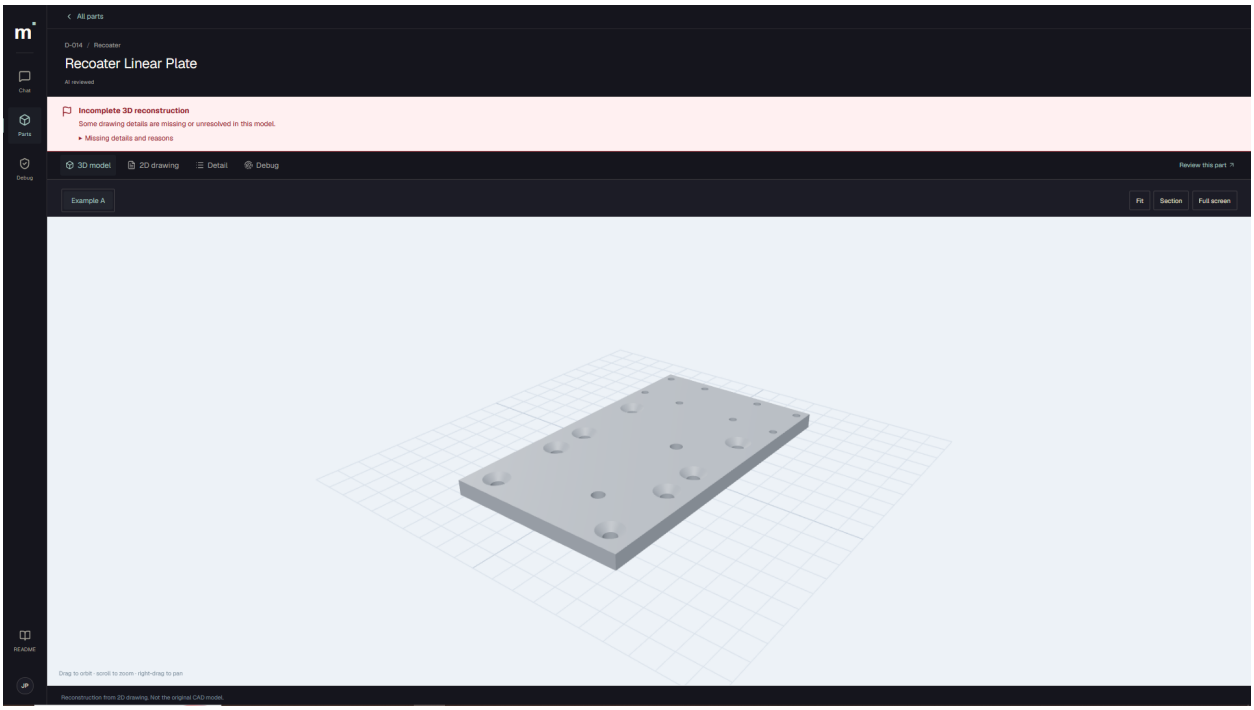


Figure 3. The Recoater Linear Plate's reconstructed 3D model, with its countersunk holes.

Because the models are generated, I wanted an independent way to tell when one is badly off. The system multiplies each model's volume by its material's density and compares the result to the weight printed on the drawing. A big mismatch usually means the thickness or scale is wrong. That makes the weight check useful for choosing between two thickness readings too, so its result is shown next to the switch.

The other 21 drawings don't have models yet, mostly because they're bent sheet metal or other shapes the flat-part template can't describe. To get a sense of how much room there is to grow, I had a separate model estimate how many of the 30 drawings a CAD engineer could model, and it said 23, which puts the current nine at about 39%. More templates would close most of that gap, and since the pipeline already reads the drawings, each new template mainly needs its CadQuery code.

Corrections and review

Even with careful preparation, some data will be wrong, either because the source was wrong to begin with or because the models misread it. Corrections give an engineer a way to fix that without losing the original. When a user tells chat that something is wrong, chat creates a pending correction on a review page, and a reviewer decides whether to accept it.

D-019, a silicone gasket, was the test case. This is the drawing whose title field says “Description,” but its filename, its Danish note, and its linked BOM row all point to “Silicone gasket for f-theta lens.” Before the correction is approved, asking “What is D-019?” gives an Undetermined title. After approval, the same question gives the new title, and the part’s history still shows what the drawing originally said.

The review page is also where a person settles anything the pipeline couldn’t and approves material mappings. All of these decisions are stored separately from the prepared data, because the prepared data gets rebuilt whenever a source file changes and the decisions need to survive that. Every rebuild reapplies them, and if a drawing changes, only the decisions based on that drawing go back for review. The decisions also ship with the app, so a customer sees the engineer’s approvals without redoing them. So far, 425 of the pipeline’s proposals have been reviewed, with 424 confirmed and one rejected.

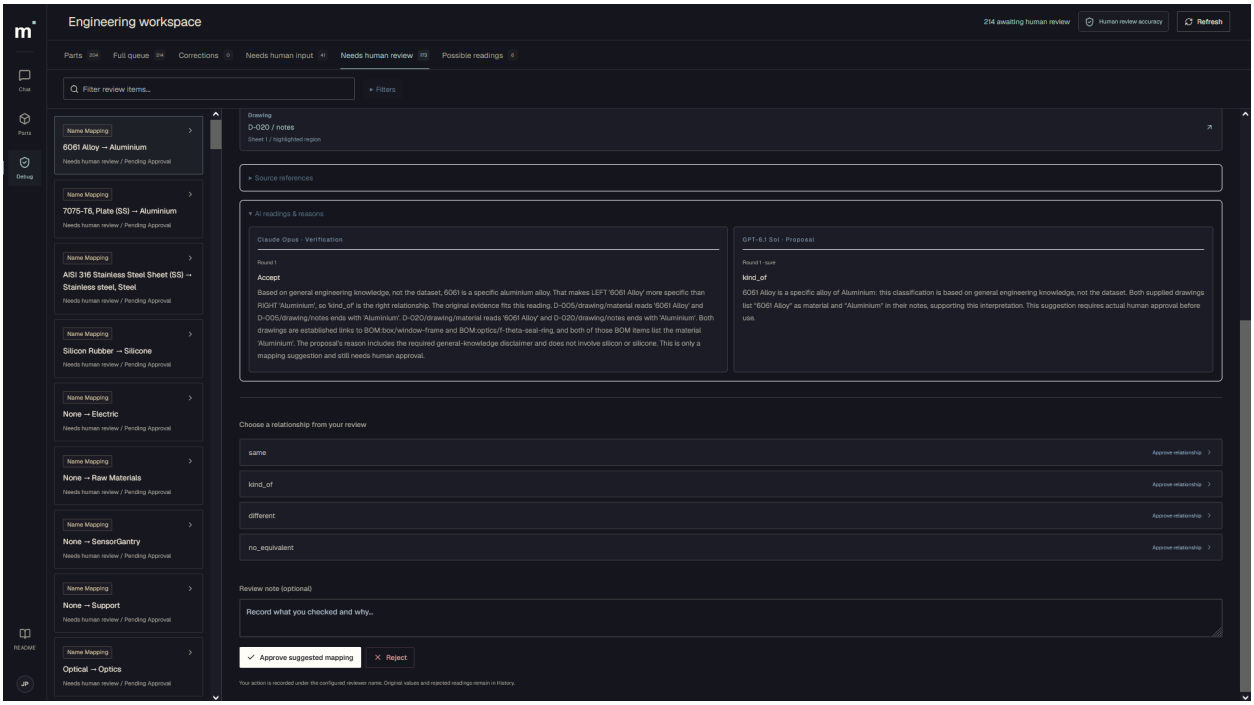


Figure 4. A material mapping on the review page, with the proposer’s suggestion and the reviewer’s verdict side by side.

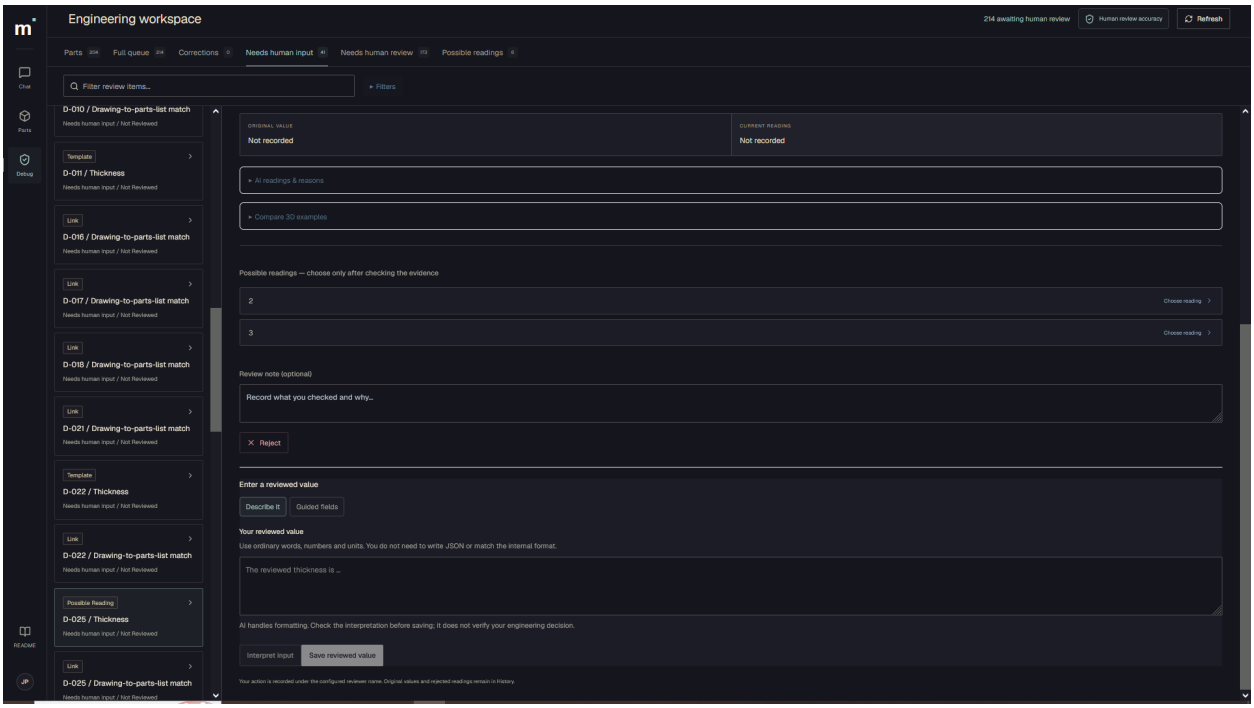


Figure 5. The Build Base Ring's thickness, printed as both 2 mm and 3 mm, waiting for a reviewer to pick one.

Interface and stack

Most people using Meridian just want an answer they can trust, while a few will want to see exactly how a value was reached. So the default view stays simple, showing a part's key values, its drawing, and whether it's been reviewed, and everything else lives in a separate Debug workspace. The parts catalog and chat use the same part view, and in chat the answer and drawing sit side by side, so you can check a value without switching pages.

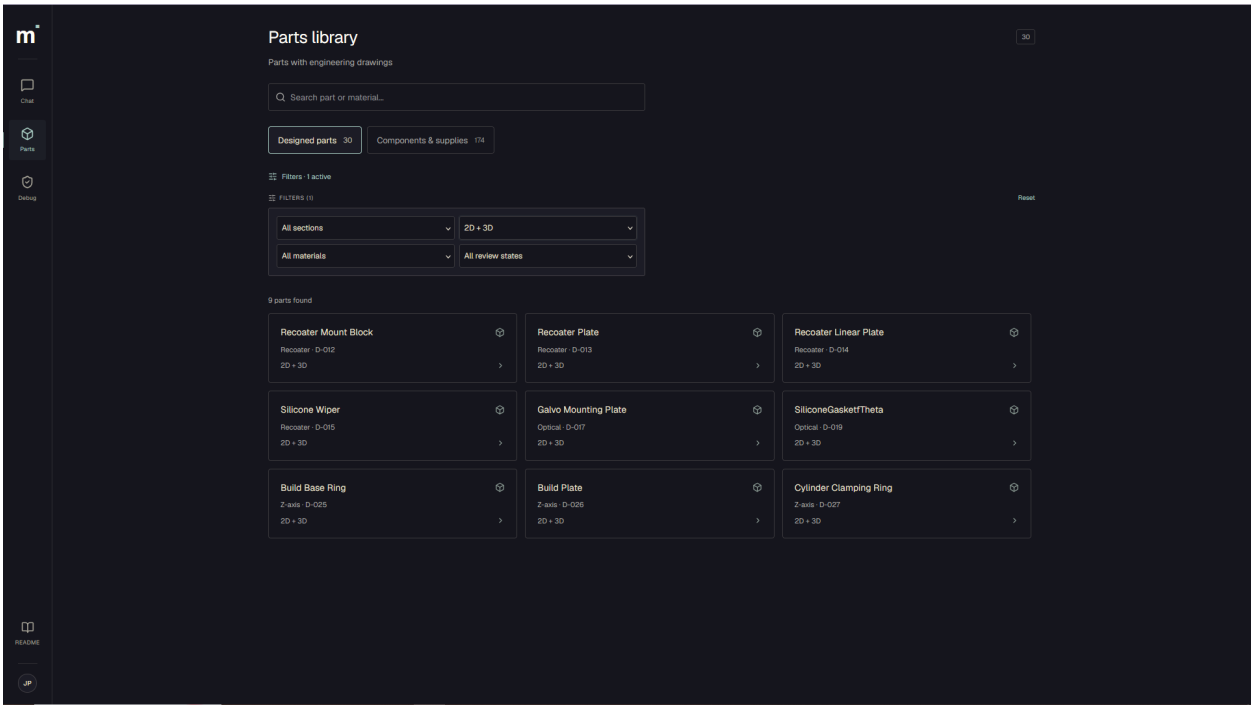


Figure 6. The parts catalog, filtered here to the nine parts that have both a drawing and a 3D model.

Under the hood, the backend is Python and FastAPI, which lets the server share a language with the extraction code and CadQuery. The frontend is React, with Three.js for the 3D viewer. The dataset is small, so the prepared records are just JSON files loaded into memory. Everything the preparation stage produces ships with the app, so running it only takes Python.

Both stages send data to outside AI providers. Preparation sends drawing images to OpenAI and Anthropic, and chat sends each question and the facts it looked up to whichever provider is configured. I decided that was acceptable because companies using AI tools usually go through approved enterprise accounts, so what matters is sticking to approved providers and being clear about what gets sent.

Testing and benchmark

I used testing to validate the design and to see how Meridian holds up in real use. That took two kinds of testing: automated tests that check each part of the system does what it’s supposed to, and a benchmark that uses the whole system the way an engineer would.

The automated tests focus on the failures that would mislead an engineer, such as a citation pointing to the wrong spot on a drawing. The latest full run passed 586 tests. For the 3D models, the tests probe 1,354 points inside the solids to check whether material is where it should be, which catches a missing hole that a simple “does the file exist” check would never notice.

The benchmark is 20 questions an engineer could plausibly ask, written from the source files, including some the data can't fully answer. One question is asked before and after the D-019 correction, to check that an approved fix actually changes the answer. I ran the full set twice to see whether the answers stayed consistent, giving 42 answers in total. A separate Claude Opus judge checked whether each answer meant the right thing, and automatic checks confirmed the citation and Undetermined markers were used correctly. The score was 38 of 42, or 90.5%, and both runs gave the same result on every question.

The judge found the meaning correct on all 42 answers, so every failure came from the markers. The four failures were the same two mistakes in both runs. In one, the model correctly said the material on the Build Plate's drawing was unclear, but cited it like a known fact. In the other, it correctly said no screw torque was given, but also marked the known M5 hole spec as Undetermined. The tools returned the right information both times, so the fix is in how the chat prompt explains the markers.

An earlier run turned up a different weakness: the model sometimes miscounted parts even when the tools gave it the complete list. Code counts more reliably than a language model, so the next step is to have the tools return counts directly.

Speed and cost

The median response took 4.3 seconds, and 95% came back within 12.2 seconds. The benchmark used about 940,000 chat tokens, almost all of it the tool results sent to the model. Chat works with either OpenAI or Anthropic models.